

ELASTICO

Football Analytics

Platform

Architecture, ML pipeline, design system audit, data integrity verification, and blocker analysis for the full-stack football prediction system.

Author: Z.ai

Date: August 25, 2026

Version: 0.2.0

Classification: Internal Technical Report

Table of Contents

1. Executive Summary	3
2. Technical Architecture	3
2.1 Technology Stack	3
2.2 Application Structure	4
2.3 Data Providers and Integration	4
2.4 Database Schema	5
3. Design System and UI/UX	5
3.1 Design Tokens and Foundation	5
3.2 Navigation and Shell	5
3.3 Design Decision Audit	6
3.4 Data Honesty and Zero-Fabrication	6
4. Machine Learning Pipeline	6
4.1 Training Data	6
4.2 Model Architecture and Performance	7
4.3 Kelly Criterion and Market Integration	7
5. Data Integrity and Verification	7
5.1 Environment Status	7
5.2 Understat xG Pipeline Blocker	8
5.3 UI Data Handling Issues	8
5.4 Database State Summary	9
6. Feature Inventory	9
7. Known Issues and Blockers	9
7.1 Critical Blockers	9
7.2 UI Data Display Issues	10
7.3 ML Pipeline Overfitting	10
8. Recommendations	10

8.1 Immediate (Unblock Data Pipeline) 10

8.2 Short-Term (Data Quality) 10

8.3 Medium-Term (ML Improvement) 11

9. Build and Deployment 11

10. Conclusion 11

1. Executive Summary

ELASTICO is a full-stack football (soccer) analytics and prediction platform built on Next.js 16, React 19, Prisma 6, and PostgreSQL. The system integrates six external data providers (ESPN, football-data.org, TheSportsDB, The Odds API, Understat, and Newsdata.io) to deliver match intelligence, expected goals (xG) analysis, tactical visualizations, AI-powered chat, and a Python-based machine learning prediction engine. The platform comprises 171 TypeScript source files totaling 41,769 lines of code, with 50 API routes, 21 feature views, 48 shadcn/ui components, and 8 design-system primitives. A companion Python ML pipeline (29 modules) implements XGBoost, LSTM, ensemble, and Kelly Criterion engines trained on 7,537 matches.

The project has undergone 14 major development phases documented in 76 design decisions (DS-001 through DS-067), progressing from raw feature development through a rigorous design-system foundation, asset resolution pipeline, shell/navigation redesign, dashboard and live-matches rebuild, full DS compliance audit across all views, and a data-truth audit that eliminated fabricated defaults, silent catch blocks, and null-to-zero transforms. The codebase compiles with zero TypeScript errors and zero **as any** casts in any view file.

Critical Blocker (Cycle 4.5): Real expected-goals data cannot be populated. The Understat `getTeamData` endpoint returns HTTP 404 for all team IDs, blocking the entire xG computation pipeline. Additionally, `API_SPORTS_KEY` and `NEWSDATA_API_KEY` are unconfigured, preventing entity resolution and news sync. Zero `TeamAnalytic` rows and zero real xG values exist in the production database.

2. Technical Architecture

2.1 Technology Stack

The frontend is a single-page application built with Next.js 16.1.1 (App Router) and React 19, styled with Tailwind CSS 4 and shadcn/ui. The data layer uses Prisma 6.19.3 with PostgreSQL (Neon) as the primary database, while Zustand manages client-side state. The application deploys to Vercel with standalone output mode, generating 33 static pages and 51 dynamic API routes at build time. The production build compiles in approximately 22.6 seconds with zero TypeScript errors, zero schema errors, and zero import errors.

Layer	Technology	Version	Purpose
Framework	Next.js	16.1.1	App Router, SSR, API routes
UI Library	React	19.0.0	Component rendering
Styling	Tailwind CSS	4.x	Utility-first CSS
Component System	shadcn/ui	latest	48 UI primitives
Database ORM	Prisma	6.19.3	Type-safe data access

Database	PostgreSQL (Neon)	--	Primary data store
State	Zustand	--	Client-side state
Charts	Recharts	--	Data visualization
Deployment	Vercel	--	Standalone output mode
ML Pipeline	Python 3	3.12+	XGBoost, LSTM, ensemble

Table 1: Core technology stack

2.2 Application Structure

The codebase follows a layered architecture with clear separation of concerns. The `src/app/api/` directory contains 50 API route files organized by domain (matches, teams, players, predictions, analytics, admin, auth, sync, etc.), each handling a single REST endpoint. The `src/components/elastico/` directory holds 21 view-level components (e.g., `dashboard-view.tsx`, `matches-view.tsx`, `tactical-view.tsx`), each responsible for a full screen of the application. Shared reusable components live in `src/components/ui/` (48 files) from `shadcn/ui`, while domain-specific design primitives (`TeamCrest`, `PlayerHeadshot`, `FlagIcon`, `StatusBadge`, etc.) are isolated in `src/components/elastico/primitives/` (8 files). Business logic, data transformations, and external API integrations are encapsulated in `src/lib/` (30+ modules totaling 10,930 lines), covering everything from the design system token definitions to entity resolution, prediction engines, and provider-specific API clients for each data source.

Directory	Files	Lines	Role
src/components/elastico/	21 views + 10 other	15,539	Feature screens, shell, primitives
src/lib/	30+ modules	10,930	Business logic, API clients, engines
src/app/api/	50 routes	8,693	REST API endpoints
src/components/ui/	48 components	~6,000	shadcn/ui design system
src/store/	1 module	523	Zustand global state
src/hooks/	2 modules	353	Custom React hooks
football-prediction-mega/	29 Python files	~4,500	ML training pipeline

Table 2: Codebase structure and size

2.3 Data Providers and Integration

ELASTICO integrates six external data providers, each serving a distinct role in the data pipeline. ESPN provides the primary match, team, and standings data through public API endpoints, and serves as the most reliable source for team logos and league badges. `football-data.org` offers structured match and competition data with a configured API key. `TheSportsDB` supplements with player images, team badges, and player biographical data. The Odds API delivers pre-match and live betting odds for the prediction engine and Kelly Criterion calculations. Understat provides expected goals (xG) data for tactical analysis, though as of the Cycle 4.5 verification, the `getTeamData` and `getMatchData` endpoints return HTTP 404, blocking the entire

xG pipeline. Newsdata.io is intended to supply football news articles, but its API key is unconfigured, preventing any news data from being synced.

A canonical entity resolution system (CanonicalTeam + SourceIdentity models in Prisma) was designed to unify team identities across these providers. However, this system has never been populated in the production database because the API-Sports sync (which triggers entity resolution) has never completed successfully due to the missing `API_SPORTS_KEY`. Currently, 670 teams exist in the database, but zero have been linked through the canonical entity model. Asset resolution for team crests, player headshots, and national flags follows a priority chain defined in DS-011 through DS-017, with ESPN logos preferred for crests, TheSportsDB cutouts for headshots, and flagcdn.com for national flags, all with graceful fallbacks.

2.4 Database Schema

The Prisma schema defines 23 models covering the full domain: Team, Match, Player, Standing, Prediction, TeamAnalytic, OddsSnapshot, NewsArticle, SyncLog, User, Session, Announcement, FeatureFlag, Bookmark, Achievement, UserAchievement, Notification, Subscription, CanonicalTeam, SourceIdentity, and provider-specific models. The Match model supports full provenance tracking for xG data with `homeXg`, `awayXg`, `homeXgSource`, `awayXgSource`, `homeXgTruthClass`, and `awayXgSource` fields. The TeamAnalytic model supports `truthClass` (REAL, DERIVED, SIMULATION, DEMO), `dataFreshness` classification (FRESH, CURRENT, SEASON, STALE), and source tracking. Currently, the database holds 670 teams, 27 matches, 192 standings rows, 15 sync logs, and zero rows in TeamAnalytic, Player, OddsSnapshot, and NewsArticle tables.

3. Design System and UI/UX

3.1 Design Tokens and Foundation

The design system is codified in `src/lib/design-system.ts` as the single source of truth, eliminating magic numbers scattered across 26 views. It defines a 10-level typography scale (display through monoSm), a 4px-base spacing system with 7 named presets (xs through 2xl), a 4-level surface hierarchy (base through sunken), and 8 size presets for assets (xs=16px through 3xl=64px). All chart theming is unified through `src/lib/chart-theme.ts`, which provides consistent `axisProps`, `cartesianGridProps`, `tooltipContentStyle`, and a cycling 8-color palette (DS-006, DS-007). Match status rendering follows a 5-state model (live, halftime, finished, upcoming, postponed) with specific color, background, border, and pulse configurations (DS-008). Form colors follow universal football conventions: W=emerald, D=amber, L=red (DS-009). Data classification uses 5 badges (REAL, DERIVED, SIMULATION, DEMO, BUG) enforcing the data honesty principle across all views (DS-010).

3.2 Navigation and Shell

The sidebar navigation is organized by football workflow rather than technical category (DS-022): Intelligence (Dashboard, Live Matches, Match Analysis), Analysis (Tactical, Players, Compare, Predictions, Prediction Engine), Leagues (Standings, Leaderboard), Tools (AI Chat, News, Export, Social), and System

(Settings, Notifications, Subscription, Admin, System Monitor, Achievements). All 22 views are mapped in the sidebar with no orphaned screens. The command palette (Cmd+K) covers all 19 navigable views, grouped by workflow when no search query is active. The header was stripped to a minimal context bar, removing zoom controls and plan badge (DS-028), and view titles were standardized (e.g., match-detail became Match Analysis, tournament became Standings). Keyboard shortcuts were realigned: Cmd+T opens Tactical, Cmd+S opens Standings.

3.3 Design Decision Audit

Over 14 development phases, 67 formal design decisions (DS-001 through DS-067) were recorded in the living `DESIGN_STATE.md` document. These decisions span five major categories: Design System Foundation (DS-001 to DS-010, 10 decisions covering tokens, typography, spacing, surfaces, charts, colors, status, form colors, data classification), Asset System (DS-011 to DS-021, 11 decisions covering resolution priority chains, caching, fallbacks, and the no-raw-img rule), Shell/Navigation Redesign (DS-022 to DS-038e, 17 decisions covering workflow groups, header minimalism, command palette, grid alignment, and footer data credits), Dashboard and Feature Screen Rebuilds (DS-039 to DS-062, 24 decisions covering component compliance, chart theme enforcement, data classification badges, and zero-fabrication rules), and Data-Truth Audit (DS-063 to DS-067, 5 decisions covering error logging, fabricated default removal, and null-coalescing fixes). Each decision records the component or area affected, the specific choice made, and the rationale.

3.4 Data Honesty and Zero-Fabrication

A core architectural principle, enforced through Phase 14, is that the UI must never present fabricated data as real. This was implemented through three major changes. First, 20 silent catch blocks across 9 views were converted to `log console.error` or `console.warn` (DS-065), ensuring errors reach the debugging surface rather than being silently swallowed. Second, fabricated defaults were removed from admin-view: the old code used `|| 28` for `proCount`, `|| 12` for `eliteCount`, and `|| 24` for `matches`, which presented fake numbers as real metrics (DS-066). Third, 25 instances of `|| 0` were replaced with `?? 0` (for mathematical operations) or `?? '--'` (for display) across admin, player, and system-monitor views (DS-067), preventing falsy values like empty strings from masquerading as zero. The data classification badge system (DS-010, DS-061) now covers predictions, tournament, leaderboard, and prediction-engine views, with every data-displaying view showing REAL, DERIVED, SIMULATION, or DEMO badges alongside its metrics.

4. Machine Learning Pipeline

4.1 Training Data

The ML pipeline operates on a processed dataset of 7,537 football matches stored in both CSV (5.4 MB) and Parquet (1.6 MB) formats. The data was prepared through a multi-phase pipeline: Phase 2 established the data foundation with quality checks, Phase 2.5 added a validation gate, and Phase 3.5 subjected the pipeline to stress testing. The feature schema is defined in `football-prediction-mega/src/ml/features.py`, covering match-level features (home/away team statistics, form, league position) and engineered features derived from

historical performance data. The training pipeline (training_pipeline.py) handles data splitting, feature engineering, and model training orchestration.

4.2 Model Architecture and Performance

The prediction system employs an ensemble approach combining multiple model architectures. The primary model is XGBoost (xgboost_engine.py), trained on the full feature set with three-class classification (Home Win, Draw, Away Win). On the training set (3,657 samples), XGBoost achieves 85.7% accuracy with macro F1 of 0.858, but validation performance (1,219 samples) drops to 50.9% accuracy with macro F1 of 0.465, indicating significant overfitting. The Expected Calibration Error (ECE) on validation is 0.040, suggesting reasonable probability calibration despite the accuracy gap. The LSTM engine (lstm_engine.py) processes sequential match data using sliding windows (build_sequences.py), with LSTM metrics recorded in phase4_lstm_metrics.json. The ensemble engine (ensemble.py) combines XGBoost and LSTM predictions using weighted averaging, with the stochastic engine adding controlled randomness for Monte Carlo uncertainty estimation.

Model	Split	Accuracy	Macro F1	ECE
XGBoost	Train (n=3,657)	85.7%	0.858	0.192
XGBoost	Validation (n=1,219)	50.9%	0.465	0.040
LSTM	See phase4_lstm_metrics.json	--	--	--
Ensemble	Weighted average	--	--	--

Table 3: ML model performance summary

4.3 Kelly Criterion and Market Integration

The prediction engine includes a Kelly Criterion implementation (kelly.py) that calculates optimal stake sizes based on model-predicted probabilities and available market odds. The market signals engine (market_signals.py) detects value bets by comparing model probabilities against bookmaker odds, identifying situations where the model believes the market has mispriced a match outcome. The frontend prediction-engine view provides interactive simulation controls, Kelly fraction visualization, and a comparison of model vs. market probabilities. The calibration analysis (Phase 5) produced calibration metrics and reliability diagrams (phase5_plots/ directory) to assess how well model probabilities match actual outcome frequencies across different confidence bins.

5. Data Integrity and Verification

5.1 Environment Status

The Cycle 4.5 verification (August 23, 2026) performed a comprehensive end-to-end audit of the production environment. Of the four external API keys required, two are configured (FOOTBALL_DATA_API_KEY, THE_ODDS_API_KEY) and two are empty (API_SPORTS_KEY, NEWSDATA_API_KEY). The database connection is verified with 670 teams in the database. Understat's getLeagueData endpoint responds with

HTTP 200, confirming basic reachability, though the deeper `getTeamData` and `getMatchData` endpoints return 404.

Service	Status	Impact
FOOTBALL_DATA_API_KEY	CONFIGURED	Match data operational
THE_ODDS_API_KEY	CONFIGURED	Odds data operational
API_SPORTS_KEY	NOT CONFIGURED	Blocks team sync, entity resolution
NEWSDATA_API_KEY	NOT CONFIGURED	Blocks news sync
DATABASE_URL	CONFIGURED	670 teams, 27 matches
Understat (HTTP 200)	PARTIAL	League data works, team/match 404

Table 4: External service configuration status

5.2 Understat xG Pipeline Blocker

The most critical finding from the Cycle 4.5 verification is the complete failure of the Understat xG pipeline. While the `getLeagueData` endpoint successfully returns 20 Premier League teams for the 2024/25 season with valid IDs and names, and all three tested teams (Arsenal ID=164, Chelsea ID=167, Liverpool ID=228) resolved correctly through the entity resolution system, the `getTeamData` endpoint returns HTTP 404 for every team ID tested. This means the `computeTeamXgFromMatches()` function cannot execute, as it depends on match-level xG data that is only available through this failed endpoint. The `getPlayersStats` POST endpoint does work (e.g., Salah xG=27.71), but the current pipeline architecture does not use player-level xG for team-level computation, making this a new development task rather than a quick fix.

The downstream consequences are severe: zero `TeamAnalytic` rows exist in the database, zero matches have real xG values (10 matches have xG=0 from demo/seeded data with null provenance), and the canonical entity model (`CanonicalTeam` + `SourceIdentity` tables) remains completely empty because entity resolution depends on API-Sports sync, which cannot proceed without an API key. The tactical view, match detail view, and compare view all have xG display logic, but every xG value shown is either 0 (from demo data) or '--' (from null), never a real Understat-sourced value.

5.3 UI Data Handling Issues

The Cycle 4.5 verification identified eight specific locations where the UI silently transforms null/missing xG data into displayed zero values, creating a fabrication risk. In `match-detail-view.tsx`, six instances of `'homeXg ?? 0'` and `'awayXg ?? 0'` cause the xG timeline chart to display a flat line at 0 and the match header to show 'xG 0.0' when no xG data exists, which a user could reasonably interpret as real data. In `tactical-view.tsx`, two instances of `'xgPerGame ?? 0'` display '0.00' instead of '--' or 'MISSING' when team xG data is unavailable. These are not fabrications in the sense of generating fake data, but they represent a form of silent data falsification: the UI displays 'xG 0.0' when the truthful state is 'xG MISSING'. The compare view correctly uses `'?? null'` and displays '--' with a clear 'No xG data available' message, serving as the reference implementation for proper null handling.

5.4 Database State Summary

The production database contains 670 teams, 27 matches (10 demo/unknown source with xG=0 defaults, 17 from API-Sports with null xG), 192 standings rows, 15 sync logs, and zero rows in Player, TeamAnalytic, OddsSnapshot, and NewsArticle tables. All unique constraints are enforced with zero duplicates. The zero Player rows reflect the fact that API-Sports player sync has never completed. The zero OddsSnapshot rows indicate that The Odds API sync has never been triggered despite the key being configured. The 15 sync logs document attempted synchronization runs, with the most recent ones reflecting the Understat 404 errors.

6. Feature Inventory

ELASTICO provides 21 distinct feature views organized into five workflow groups. Each view is a self-contained React component that fetches its own data through dedicated API routes. The Intelligence group (Dashboard, Live Matches, Match Analysis) forms the core match-day workflow, providing at-a-glance KPIs, a live match ticker, clickable match cards, and a detailed match analysis screen with xG timelines, team statistics, and possession charts. The Analysis group (Tactical Board, Players, Compare, Predictions, Prediction Engine) supports deep-dive investigation with tactical formations, player profiles and headshots, side-by-side team comparison, prediction history tracking, and an interactive Kelly Criterion simulator.

The Leagues group (Standings, Leaderboard) provides competition-wide context with league tables and user prediction rankings. The Tools group (AI Chat, News, Export, Social) offers an AI-powered football assistant, a news feed with category filtering, CSV/PDF data export, and social features. The System group (Settings, Notifications, Subscription, Admin, System Monitor, Achievements) handles application management with user preferences, notification center, subscription tiers, admin user management with RBAC, real-time system health monitoring, and a gamification system. The dashboard was redesigned in the most recent cycle as a command center with a KPI strip, live ticker, and asymmetric 2:1 split layout featuring a primary match panel and a compact news rail.

7. Known Issues and Blockers

7.1 Critical Blockers

The following issues are classified as blockers because they prevent core functionality from operating. Each blocker has a clear root cause and requires specific external action or development work to resolve.

ID	Issue	Root Cause	Resolution Required
B1	No real xG data	Understat getTeamData 404	New xG source or API fix
B2	No entity resolution	API_SPORTS_KEY empty	Configure API-Sports key
B3	No news data	NEWSDATA_API_KEY empty	Configure Newsdata.io key

B4	No odds snapshots	Sync never triggered	Implement odds sync cron
----	-------------------	----------------------	--------------------------

Table 5: Critical blockers

7.2 UI Data Display Issues

Eight locations in the codebase silently transform null xG data into zero, which misleads users into believing real data exists. These should be changed to display '--' or a 'No data' indicator consistent with the compare view's implementation. Additionally, 10 demo matches from the 'unknown' source have explicit xG=0 values with null provenance fields (homeXgSource, awayXgSource, homeXgTruthClass, awayXgTruthClass all null), which should be cleaned to either null xG or properly tagged as DEMO data.

7.3 ML Pipeline Overfitting

The XGBoost model exhibits significant overfitting: training accuracy of 85.7% drops to 50.9% on validation (a 34.8 percentage-point gap). The macro F1 drops from 0.858 to 0.465. This suggests the model is memorizing training patterns rather than learning generalizable features. Potential mitigations include increased regularization (max_depth reduction, min_child_weight increase), feature selection to reduce dimensionality, cross-validated hyperparameter tuning, and incorporating domain-specific features that capture more fundamental aspects of football match outcomes rather than surface-level statistics.

8. Recommendations

8.1 Immediate (Unblock Data Pipeline)

- **Configure API_SPORTS_KEY:** This single action unblocks team/player sync, canonical entity resolution, and the entire data provenance chain. Without it, the 670 teams in the database cannot be linked to their identities across providers.
- **Configure NEWSDATA_API_KEY:** Restores the news pipeline, which provides contextual match previews and editorial content for the dashboard news rail and dedicated news view.
- **Fix Understat xG pipeline or find alternative source:** The getTeamData 404 may be a temporary API change or a permanent deprecation. Investigate whether the playersStats endpoint can be aggregated for team-level xG, or evaluate alternative xG providers (e.g., FotMob, Sofascore, xG by Opta via fbref scraping).
- **Implement odds sync cron job:** The Odds API key is configured but the sync has never been triggered. A scheduled job should fetch pre-match odds for upcoming fixtures and store them as OddsSnapshot rows.

8.2 Short-Term (Data Quality)

- **Fix 8 null-to-zero xG transforms:** Replace '?? 0' with '?? null' in match-detail-view.tsx (6 locations) and tactical-view.tsx (2 locations), then display '--' when null, consistent with the compare view.

- **Clean demo match xG data:** Set xG to null for the 10 'unknown' source matches that have explicit 0 values, and add proper truthClass=DEMO tagging.
- **Populate canonical entity model:** Once API_Sports sync is operational, run the entity resolution pipeline to link all 670 teams across providers. This enables accurate cross-provider data merging.

8.3 Medium-Term (ML Improvement)

- **Address XGBoost overfitting:** Implement cross-validated hyperparameter search with reduced max_depth (try 3-5 instead of default 6), increased min_child_weight, and L1/L2 regularization. Target a validation accuracy above 55% with a train-validation gap below 15 percentage points.
- **Add feature importance analysis:** Use SHAP values to identify which features drive predictions and remove noise features that contribute to overfitting.
- **Explore alternative xG sources for model features:** Once real xG data flows into the database, retrain models with xG-based features which are more predictive than raw goal statistics.

9. Build and Deployment

The production build uses Next.js standalone output mode, generating 33 static pages and 51 API routes. The build completes in approximately 22.6 seconds with zero TypeScript errors, zero schema errors, and zero import errors. The application deploys to Vercel with environment variables for database connection, API keys, and auth secrets. The most recent commit (1c2e341) fixed match analysis navigation by adding the fd: prefix to football-data.org IDs and redesigned the dashboard as a command center. The build has been verified to pass after each of the 14 development phases, with the design system compliance changes (DS-055 through DS-067) maintaining the zero-error build standard throughout.

The deployment configuration includes a Caddyfile for reverse proxy, a docker-compose.yml for containerized development, and a Vercel configuration for production. The PWA manifest and service worker (sw.js) are present in the public directory, indicating planned offline/mobile support. Security verification confirmed that no API key values are exposed in client-side code; only key names appear in settings and system monitor documentation text.

10. Conclusion

ELASTICO represents a substantial engineering achievement: a full-stack football analytics platform with 41,769 lines of TypeScript, 50 API routes, 21 feature views, a Python ML pipeline with multiple model architectures, and a rigorously documented design system with 67 formal decisions. The codebase quality is high, with zero TypeScript errors, zero type-unsafe casts in views, unified chart theming, consistent data classification badges, and a strict no-fabrication principle enforced through dedicated audit phases.

However, the platform's analytical value is fundamentally constrained by the data pipeline blockers identified in Cycle 4.5. Without real xG data, the tactical analysis, match detail, and prediction features operate on

empty or demo data. Without API-Sports sync, the canonical entity model remains unpopulated, preventing accurate cross-provider data merging. The ML models show promising calibration (ECE 0.040) but suffer from significant overfitting that limits real-world predictive value. Addressing these blockers, particularly the xG data source and API key configuration, should be the immediate priority to unlock the platform's full potential.